

TinySAM 论文阅读笔记

论文信息

- 标题: TinySAM: Pushing the Envelope for Efficient Segment Anything Model
- 作者: Han Shu, Wenshuo Li, Yehui Tang, Yiman Zhang, Yihao Chen, Houqiang Li, Yunhe Wang*, Xinghao Chen*
- 机构: 中国科学技术大学 & 华为诺亚方舟实验室
- 会议: AAAI 2025

因为计算机科学技术发展前沿的大作业来拜读本篇论文。选它主要是看到 AAAI 2025 的论文比较新，而且 SAM (Segment Anything Model) 这个东西之前听说过，一直觉得挺神奇的——给张图，点一下，它就能把你想要的那个物体完整地圈出来。SAM 有个很棘手的问题：太大、太慢了。TinySAM 标题里说要 "push the envelope" (突破极限)，那么它到底怎么把这么一个庞然大物变小？

理解的这篇论文要解决什么问题

SAM 是 Meta 做的一个图像分割大模型。简单说就是，你随便给它一张图，然后用鼠标点一下或者画个框，它就能自动把那部分精确地分割出来。这个能力很强，所以 SAM 被用到了很多地方，比如视频里的目标追踪、医学影像分析等等。

但 SAM 的问题也很明显——太吃计算资源了。它最大的那个版本用的是 ViT-H 这种重量级的图像编码器，处理一张正常大小的图片就要花差不多两秒，而且这还是在一张好的显卡上。放到手机或者嵌入式设备上根本跑不动。

其实之前已经有人想过把 SAM 做小了。我读论文的时候了解到，有个叫 MobileSAM 的工作把图像编码器换成了轻量的 TinyViT，然后用了一种叫 "蒸馏" 的技术，让这个小模型去模仿大模型的输出。这个思路挺好，但作者指出它有个缺陷：MobileSAM 只让小模型模仿图像编码器那一层的输出，至于后面 mask decoder 怎么一步步推理出最终结果，小模型完全没有学过。就像老师只告诉你 "看到了什么"，但没告诉你 "看到之后应该怎么想"。

TinySAM 的出发点就是：能不能在一个统一的框架里，把 SAM 从训练到推理全流程都做小、做快？

这篇论文到底做了哪些事

TinySAM 的方法分成三块，我的理解。

第一块：训练阶段——“全阶段蒸馏”加两个技巧

先说说蒸馏是什么意思。我的理解：你有一个很厉害的大模型（叫"教师"），还有一个很小的模型（叫"学生"）。蒸馏就是让学生去模仿教师的行为，这样学生虽然参数少得多，也能学到大模型的本事。

MobileSAM 的蒸馏只做了一步——让学生模仿教师的图像编码器输出。但作者觉得这远远不够。他们提出了一种"全阶段"蒸馏，在三层同时施加监督：

- **嵌入层蒸馏**：对齐图像编码器的输出，这是最基础的
- **token 层蒸馏**：对齐 mask decoder 中间层的输出。这时候 prompt 信息和图像信息已经发生了交互，我觉得可以理解成"解题的中间步骤"
- **输出层蒸馏**：直接对齐最终的 mask 预测结果，这是最接近最终答案的一层

读完这段大概理解了作者的想法：让学生不光模仿老师"看到了什么"，还要模仿"怎么推理的"以及"最终答案是什么"。这确实比 MobileSAM 的做法完整得多。

光有全阶段蒸馏还不够，作者还加了两个技巧。

第一个叫**硬提示采样**。训练 SAM 的时候需要随机在图上选一些点作为 prompt（提示）。普通做法是在标注 mask 范围内随机取点。但这个做法的缺点是——大部分点会落到物体中心区域，而这些区域往往很容易分割。真正难的是物体的边缘和边界，模糊、不规则，模型学不到足够多的"硬骨头"。作者的做法是：第一轮正常预测，然后在"标注有但预测没有"的区域重新采样下一轮的点。如此反复几轮，采样点会越来越靠近边界。理解这就是一种"哪里不会练哪里"的策略。

第二个叫**硬掩码加权**。数据集里不同图片分割难度是不一样的。一面纯色的墙很容易，一只毛茸茸的猫边缘就很难。如果所有样本在损失函数里权重一样，简单样本（数量又多）会把训练信号稀释掉。作者的做法是根据学生和教师预测的 IoU 差距来给每个样本加权——学生表现越差、和教师差距越大的样本，权重越高。这相当于告诉模型："这些你做不好的，多练几遍。"

第二块：部署阶段——把模型量化成整数

蒸馏让模型变小了，但参数还是浮点数。要上移动端，浮点运算吃资源，最好是变成 8 位甚至 6 位的整数运算。

量化这件事本质上就是做一次舍入——把连续的数值映射到离散的格点上。但问题在于：不是所有的权重都同样重要。有些权重稍微动一点，输出就大变样；有些怎么动都没事。如果一刀切地量化，敏感的那些权重就会引入大误差。作者用了 Hessian 矩阵来评估每个权重的敏感度——敏感度高的，量化时步长设小一点，保留更多精度；不敏感的可以粗放处理。

另外还注意到一个细节：softmax 和 GELU 这两个激活函数后面的数值分布比较特殊，不是均匀的。softmax 输出集中在 0 和 1 两端，GELU 正负区间方差差

别很大。作者的处理方式是把这些激活值拆成两组，各自用独立的缩放因子量化，减少误差。

第三块：推理阶段——“先粗后细”的加速策略

SAM 有一个模式叫 **Everything Mode**（分割一切），意思是在全图铺一个 32×32 的网格，每个网格点作为 **prompt** 送进 **mask decoder**，一共要跑 1024 次。原始 SAM 的瓶颈在图像编码器上，**mask decoder** 跑 1024 次倒不是问题。但 **TinySAM** 已经把编码器换成了超轻量的 **TinyViT**，瓶颈反而转移到了 **mask decoder**——哪怕每次调用很快，1024 次加起来也很吓人。

作者的解决思路很简单：分两轮。第一轮稀疏采样（每边只取四分之一，总共只有原来的十六分之一点数），快速得到粗糙的分割结果。然后检查每个区域的置信度，如果某个区域置信度已经很高了（说明物体大、分割得好），这个区域就不再管了。第二轮在剩下的低置信度区域恢复密集采样，精细处理。

这个策略我读完觉得特别直观——就好像先眯着眼扫一遍全场，把那些一眼就能认出来的大块头搞定，然后再凑近了仔细看那些小东西和边界模糊的地方。论文给的数据是：超过一半的采样点被跳过了，推理时间差不多减半，但分割质量几乎没有下降。

实验做了哪些、结果怎么样

论文主要在 **COCO** 和 **LVIS** 两个数据集上做了零样本实例分割测试。所谓“零样本”就是模型在这两个数据集上完全没有训练过，直接用训练好的权重去跑——这是检验 **SAM** 类模型泛化能力最重要的方式。

几个我比较关注的数字：

- **TinySAM**: 计算量 42.0G MACs，在 **COCO** 上的 AP 是 42.3%
- **MobileSAM**: 同样 42.0G MACs，AP 只有 41.0%
- **Q-TinySAM（量化版）**：计算量降到 20.3G MACs，AP 还有 41.4%

同样的计算量下比 **MobileSAM** 高了 1.3 个百分点，量化之后计算量砍半还能超过 **MobileSAM**，我觉得这个结果挺有说服力的。说明全阶段蒸馏和硬挖掘策略确实起了作用。

还有一个对比让我印象比较深：**FastSAM** 用了 443G MACs（是 **TinySAM** 的十倍多），AP 却只有 37.9%。这告诉我“做小”并不意味着“做差”——方法选对了，完全可以又小又好。

层次化推理的消融实验也很有意思。Everything Mode 下，TinySAM 从 1.78 秒变成 0.93 秒，mIoU 从 0.6137 变成 0.6061——时间砍了快一半，质量只降了不到一个百分点。这说明原方案里有一半以上的计算确实是浪费在已经很清楚的大块区域上了。

消融实验还有一点值得记：三级蒸馏损失里，output loss 贡献最大，token loss 次之，embedding loss 最小。我觉得这很好理解——离最终目标越近的监督信号越直接有效。

思考

好的地方

读完这篇论文，我最大的感受是它很"完整"。不是只在某个环节做了优化，而是从训练（蒸馏）、到部署（量化）、再到推理（层次化），每一步都照顾到了。这样读完让人感觉——如果你真的想把 SAM 部署到手机上，按照这个方法走一遍就行了，虽然实际上不好说（）。

另外硬提示采样和硬掩码加权这两个技巧让我觉得作者把问题想得很清楚。它们都不是特别复杂的数学创新，但抓住了训练过程中真正影响效果的关键点：让模型把精力从简单的区域和样本上挪开，集中攻克那些真正难的地方。

层次化推理我觉得是最"优雅"的部分。不动模型权重，不用重新训练，只改变推理时的策略，就能几乎不加代价地把速度翻倍。这种零成本的改进在实际应用中价值很大。

疑惑的地方

第一，蒸馏过程需要先把教师网络在 SA-1B 数据集上的输出全部存下来。SA-1B 是一个千万级图片的数据集，虽然单张的嵌入不算太大，但全部存下来的硬盘开销应该不小。而且如果将来想换个网络，所有嵌入都得重新算。我能不能做成在线蒸馏——每次训练时现场让教师推理——虽然训练时间会变长，但至少不用那么大的存储？

第二，层次化推理里的置信度阈值 τ 是手动设的。COCO 数据集大多是自然场景图片，这个阈值可能还行。但如果换到完全不同的场景——比如细胞显微镜图像里全是密密麻麻的小物体——同一个阈值可能就不合适了。最好能让 τ 根据图像内容自动调整。

第三，量化只试到了 6 位，再往下的 4 位没做。虽然 6 位可能精度已经不行了，但如果用混合精度——敏感层高比特、不敏感层低比特——可能还能再挤一点压缩空间。

可优化的地方

一个是让层次推理的阈值能自适应。可以试着统计第一轮分割的平均置信度，根据这个动态调整 τ 。另一个是我注意到论文所有实验都是基于点和框的 **prompt**，没提文字 **prompt** 下的表现。如果 SAM 将来要做多模态的通用分割器，这部分的表现需要验证。

一些术语

知识蒸馏：让一个小模型去模仿大模型的行为。和简单的参数压缩不同，蒸馏更像教学——不是硬把大模型压扁，而是让大模型把自己解题的过程"教"给小学生。

全阶段蒸馏：MobileSAM 只模仿了图像编码器输出这一层。TinySAM 在图像嵌入、中间 token、最终 mask 三个层级都做了监督。我的理解是——信息传递的链完整了，学生才能学得像。

硬提示采样：训练时，下一轮的采样点落在"标注有但预测没有"的差集上。相当于每次只练上次犯错的地方，和错题本一个道理。

硬掩码加权：根据学生和教师 IoU 的差距来给样本加权。差距大的样本说明对学生来说更"难"，就多练。这样损失函数会自动把训练重心倾斜到困难样本上。

后训练量化：在模型训练完成之后，把浮点参数转换成低比特整数。不重新训练，只做一次性转换，所以叫"后训练"。和训练时量化（QAT）不同，PTQ 更快更省资源。

层次化推理：先粗后细的推理策略。先用稀疏点快速扫一遍，把好分割的大区域搞定；剩下的不确定区域再用密集点精细处理。类似于先看全景再盯细节。

Hessian 矩阵：损失函数对权重的二阶导数矩阵。直观上可以理解成一个"敏感度地图"——哪里的权重稍微动一下输出就剧变，哪里怎么动都无所谓。量化时参考这个地图来分配精度。

IoU（交并比）：衡量两个区域重叠程度的指标，等于两个区域的交集面积除以并集面积。在分割任务中，IoU 越接近 1，说明预测 mask 和真实 mask 越吻合。论文中用它来比较学生和教师的预测质量。